



Assignment-2

**Web Programming using
Python and JavaScript**

Submitted By:

Name: Aditya Pal

Course: B.Tech CSE

Year: 2nd Year, 4th Semester

Roll No: 2401010119

Submitted To:

Name: Dr. Mansi Kajal

❖ **AIM:** Design and develop a fully functional Event Management Website using HTML, CSS, JavaScript (front-end) and Python Flask (back-end), enabling users to create, browse, register for, and manage events through a dynamic and responsive web interface.

❖ **Tasks:**

- **Project Setup & Introduction** – Create a project folder named **event_management_website** containing the required files and folders: `app.py`, `templates/`, `static/css/style.css`, and `static/js/script.js`. Add header comments in all files (project title, student name, roll number, date). Configure the Flask application with debug mode and define the root route (`/`). Display a styled landing page with a brief description of the Event Management System.
- **Event Listing Page (HTML + Jinja2)** – Create a base template (`base.html`) with a navigation bar, header, and footer. Develop an `events.html` page that extends the base template. Display a list or grid of events using semantic HTML elements like `div` and `ul`. Each event card should show the event name, date & time, venue, and a register button. Use Flask to pass event data (Python dictionaries) to the template dynamically.
- **Styling the Website (CSS)** – Apply a global CSS reset and define a consistent color scheme and typography. Style the navigation bar, hero section, event cards, and footer. Use Flexbox or Grid layout to create responsive designs. Add hover effects, transitions, and button animations. Ensure the website is mobile-responsive using media queries.

- **Event Registration Form (Flask + JavaScript Validation)** –

Create a registration page (register.html) with fields such as full name, email, phone number, event selection, and number of tickets. Handle form submission using a Flask POST route (/register). Implement client-side validation using JavaScript (check empty fields, email format, phone number). Display success or error messages after submission using Flask flash messages or alerts.

- **Admin Panel – Add / Edit / Delete Events** – Develop an admin page (/admin) accessible using session-based authentication. Create a form to add new events (name, date, venue, description, image URL). Display all events in a table with Edit and Delete options. Implement Flask routes for adding, editing, and deleting events (/admin/add, /admin/edit/, /admin/delete/). Use redirects and flash messages to confirm actions.

- **Search, Filter & RSVP Counter (Bonus)** – Add a live search bar on the events page using JavaScript to filter events without reloading the page. Implement category or date filters. Display RSVP counts for each event using Flask session or in-memory storage. Optionally integrate Flask-SQLAlchemy for database persistence.

❖ Tech Stack Used:

- HTML,CSS,JavaScript,Python Flask and SQL

❖ CODES:

➤ PYTHON Code(app.py):

```
from flask import Flask, render_template, request, redirect, flash, session  
  
app = Flask(__name__)
```

```

app.secret_key = "secret123"

# In-memory database
events = [
    {"id": 1, "name": "Music Concert", "date": "2026-05-10", "venue":
"Mumbai", "rsvp": 0},
    {"id": 2, "name": "Tech Conference", "date": "2026-06-15", "venue":
"Pune", "rsvp": 0},
    {"id": 3, "name": "Art Exhibition", "date": "2026-04-20", "venue":
"Delhi", "rsvp": 0},
    {"id": 4, "name": "Startup Meetup", "date": "2026-07-05", "venue":
"Bangalore", "rsvp": 0},
]

# ----- ROUTES ----- #

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/events")
def events_page():
    return render_template("events.html", events=events)

@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        event_name = request.form["event"]

        for event in events:
            if event["name"] == event_name:
                event["rsvp"] += 1

        flash("Registration Successful!", "success")
        return redirect("/events")

    return render_template("register.html", events=events)

# ----- ADMIN ----- #

@app.route("/admin")
def admin():
    if not session.get("admin"):
        session["admin"] = True # simple login bypass
    return render_template("admin.html", events=events)

@app.route("/admin/add", methods=["POST"])
def add_event():

```

```

new_event = {
    "id": len(events) + 1,
    "name": request.form["name"],
    "date": request.form["date"],
    "venue": request.form["venue"],
    "rsvp": 0
}
events.append(new_event)
flash("Event Added!", "success")
return redirect("/admin")

@app.route("/admin/delete/<int:id>")
def delete_event(id):
    global events
    events = [e for e in events if e["id"] != id]
    flash("Event Deleted!", "danger")
    return redirect("/admin")

# ----- #

if __name__ == "__main__":
    app.run(debug=True)

```

➤ HTML Code(admin.html):

```

{% extends "base.html" %}

{% block content %}
<h2>Admin Panel</h2>

<form method="POST" action="/admin/add">
<input type="text" name="name" placeholder="Event Name" required>
<input type="date" name="date" required>
<input type="text" name="venue" placeholder="Venue" required>
<button type="submit">Add Event</button>
</form>

<table>
<tr>
<th>Name</th>
<th>Date</th>
<th>Venue</th>
<th>Action</th>
</tr>

{% for event in events %}

```

```

<tr>
<td>{{ event.name }}</td>
<td>{{ event.date }}</td>
<td>{{ event.venue }}</td>
<td>
    <a href="/admin/delete/{{ event.id }}">Delete</a>
</td>
</tr>
{% endfor %}

</table>
{% endblock %}

```

➤ HTML Code(base.html):

```

<!DOCTYPE html>
<html>
<head>
    <title>EventHub</title>
    <link rel="stylesheet" href="{{ url_for('static',
filename='css/style.css') }}">
    <script defer src="{{ url_for('static', filename='js/script.js')
}}"></script>
</head>
<body>

<nav>
    <h2>EventHub</h2>
    <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/events">Events</a></li>
        <li><a href="/register">Register</a></li>
        <li><a href="/admin">Admin</a></li>
    </ul>
</nav>

<div class="container">
    {% with messages = get_flashed_messages(with_categories=true) %}
        {% for category, message in messages %}
            <p class="{{ category }}">{{ message }}</p>
        {% endfor %}
    {% endwith %}

    {% block content %}{% endblock %}
</div>

<footer>

```

```
<p>© 2026 EventHub</p>
</footer>

</body>
</html>
```

➤ HTML Code(events.html):

```
{% extends "base.html" %}

{% block content %}
<h2>Events</h2>

<input type="text" id="search" placeholder="Search events..."
onkeyup="searchEvents()">

<div class="events" id="eventList">
  {% for event in events %}
    <div class="card">
      <h3>{{ event.name }}</h3>
      <p>{{ event.date }}</p>
      <p>{{ event.venue }}</p>
      <p>RSVP: {{ event.rsvp }}</p>
      <a href="/register" class="btn">Register</a>
    </div>
  {% endfor %}
</div>
{% endblock %}
```

➤ HTML Code(index.html):

```
{% extends "base.html" %}

{% block content %}
<div class="hero">
  <h1>Welcome to EventHub</h1>
  <p>Discover amazing events & register instantly!</p>
</div>
{% endblock %}
```

➤ HTML Code(register.html):

```
{% extends "base.html" %}
```

```
{% block content %}
<h2>Register</h2>

<form method="POST" onsubmit="return validateForm()">
  <input type="text" name="name" placeholder="Full Name" required>
  <input type="email" name="email" placeholder="Email" required>
  <input type="text" name="phone" placeholder="Phone Number" required>

  <select name="event">
    {% for event in events %}
      <option>{{ event.name }}</option>
    {% endfor %}
  </select>

  <input type="number" name="tickets" placeholder="Tickets" min="1">

  <button type="submit">Submit</button>
</form>
{% endblock %}
```

➤ CSS Code(style.css):

```
body {
  font-family: Arial;
  margin: 0;
  background: #f4f4f4;
}

nav {
  background: #222;
  color: white;
  display: flex;
  justify-content: space-between;
  padding: 15px;
}

nav ul {
  display: flex;
  gap: 20px;
  list-style: none;
}

nav a {
  color: white;
  text-decoration: none;
}
```



```

.hero {
  text-align: center;
  padding: 50px;
  background: linear-gradient(to right, #4facfe, #00f2fe);
  color: white;
}

.events {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}

.card {
  background: white;
  padding: 20px;
  border-radius: 10px;
  transition: 0.3s;
}

.card:hover {
  transform: scale(1.05);
}

.btn {
  display: inline-block;
  background: #007bff;
  color: white;
  padding: 5px 10px;
}

.success {
  color: green;
}

.danger {
  color: red;
}

```

➤ JAVASCRIPT Code(script.js):

```

function validateForm() {
  let name = document.forms[0]["name"].value;
  let email = document.forms[0]["email"].value;
  let phone = document.forms[0]["phone"].value;

  if (name === "" || email === "" || phone === "") {

```

```

        alert("All fields required!");
        return false;
    }

    if (!/^\\d{10}$/.test(phone)) {
        alert("Invalid phone number!");
        return false;
    }

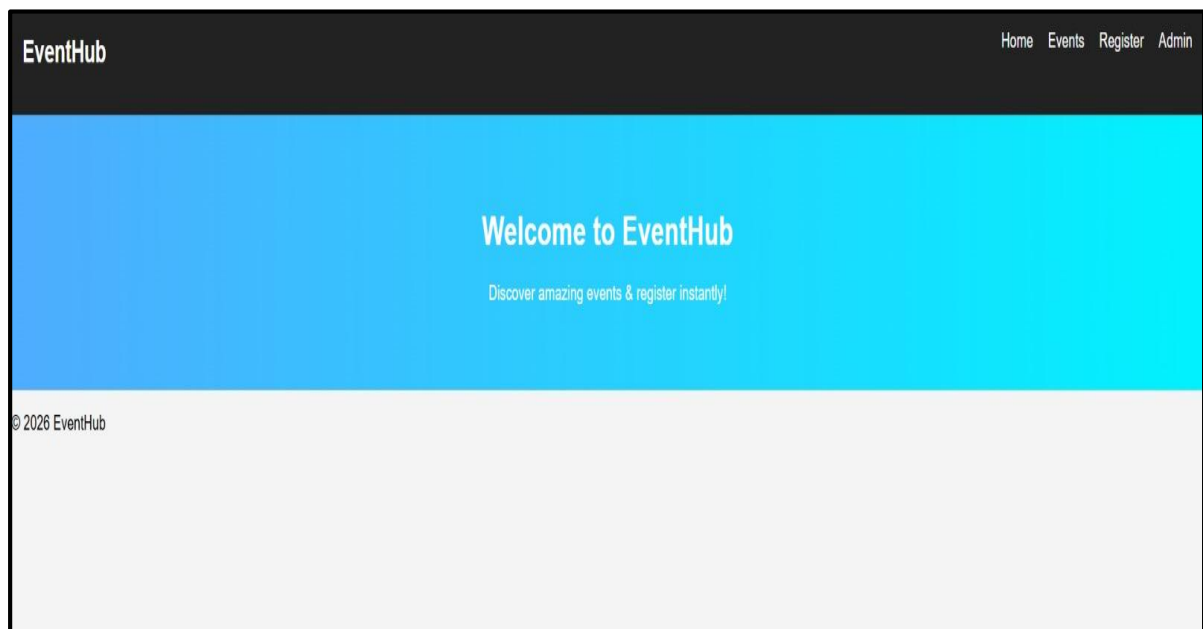
    return true;
}

function searchEvents() {
    let input = document.getElementById("search").value.toLowerCase();
    let cards = document.querySelectorAll(".card");

    cards.forEach(card => {
        let text = card.innerText.toLowerCase();
        card.style.display = text.includes(input) ? "block" : "none";
    });
}

```

❖ OUTPUT-



Events

Music Concert

2026-05-10

Mumbai

RSVP: 0

[Register](#)**Tech Conference**

2026-06-15

Pune

RSVP: 0

[Register](#)**Art Exhibition**

2026-04-20

Delhi

RSVP: 0

[Register](#)**Startup Meetup**

2026-07-05

Bangalore

RSVP: 0

[Register](#)

© 2026 EventHub